Trabajo Práctico Especial

Redes de Información (72.20)

Re-entrega

Segundo Cuatrimestre de 2026

Grupo 5

Integrantes:

Juan Bautista Albertoni Salini	64189
Santiago Devesa	64223
Junior Rambau	64461



1. Problemática y Contexto

Actualmente, la plataforma de comercio electrónico *The Store* opera sobre una arquitectura de microservicios. Si bien cada componente cuenta con endpoints para verificar su estado interno y emite registros, esta información existe de manera aislada en cada contenedor, sin tenerla centralizada en algún punto común .

Esta falta de centralización ocasiona ciertas dificultades en distintas cuestiones. En primer lugar, las métricas de salud existentes explican el “qué” falla pero no su causa raíz. Por otro lado, los logs indican el “por qué” falló una función particular pero no cuentan con el contexto global del flujo. Finalmente, la falta de trazabilidad dificulta determinar el “dónde” se originó el problema a lo largo de la cadena de llamadas entre servicios.

Teniendo esto en cuenta, en el caso de ocurrir un incidente productivo, el equipo debería revisar los logs de cada pod uno por uno e intentar sincronizar las marcas de tiempo a mano, dedicando tiempo que se resta a otras tareas. Además, fallas silenciosas o tiempos de carga demasiado largos en los flujos de la aplicación resultan en una caída en la usabilidad del producto.

2. Diseño de la solución

Continuando con lo mencionado anteriormente, con el objetivo de proteger la disponibilidad y correcto funcionamiento del sitio, se propone una solución de observabilidad unificada que combina **métricas, logs y trazas** de forma simultánea y a través de un panel.

Dado que la plataforma es desarrollada por varios equipos trabajando en paralelo, la solución se diseñó bajo una premisa fundamental: ser lo menos invasiva posible, es decir, sin modificar el código fuente de los microservicios y sin incorporar herramientas de pago que comprometan el presupuesto del proyecto.

2.1 Trazabilidad (OpenTelemetry y Grafana Tempo):

Los microservicios ya cuentan con dependencias de OpenTelemetry pero actualmente están deshabilitadas. Activarlas representaría únicamente modificar variables de entorno. Todos los servicios emitirían sus trazas vía protocolo OTLP/HTTP hacia Grafana Tempo, el cual almacena los spans sin necesidad de tener bases de datos indexadas dedicadas.

2.2 Métricas (Prometheus):

Los microservicios ya exponen sus estados en formatos nativos compatibles con Prometheus: Spring Boot lo hace a través de Micrometer en */actuator/prometheus* y los demás servicios mediante */metrics*.

Se implementa Prometheus Server bajo un modelo pull, configurando jobs de scraping periódicos configurables hacia estos endpoints y hacia la infraestructura. Esto brinda métricas RED (Rate, Errors, Duration) de cada servicio sin requerir agentes propietarios.

2.3 Logs (Promtail + Loki):

Para no cargar a las aplicaciones con librerías de logging adicionales, se despliega Promtail como DaemonSet a nivel de nodo en Kubernetes, recolectando directamente las salidas estándar (stdout) desde /var/log/pods. Promtail envía los registros a Grafana Loki.

2.4 Visualización (Grafana):

Grafana consolida todo lo mencionado en una sola pantalla. Mediante la directiva declarativa tracesToLogsV2, se vincula Tempo con Loki: al detectar un cuello de botella o un error en un span de Tempo, el operador puede abrir los logs exactos que emitió ese contenedor en esa ventana de tiempo.

3. Scope del POC y Casos de Uso

El propósito de la Prueba de Concepto es poder validar el stack de observabilidad propuesto sobre la plataforma The Store. A través de esta, se busca demostrar la eliminación de silos operativos y una posibilidad de observar incidentes mucho más veloz sin necesidad de incorporar tecnologías de pago ni editar código fuente.

3.1 Alcance:

Se trabajará sobre un clúster local de Kubernetes desplegado sobre Kind, emulando la topología de red y orquestación de un entorno distribuido. Los 5 microservicios de la plataforma (*ui, catalog, carts, orders y checkout*) serán alcanzados y se validará la funcionalidad contra los tres entornos de ejecución existentes (Java (Spring Boot), Go (Gin) y Node.js (Nest.js). Se activará la telemetría mediante variables de entorno OpenTelemetry en los manifiestos de kubernetes de cada servicio, pudiéndose modificar el kubernetes.yaml de ser necesario.

Se desplegará en un nuevo namespace (*observability*) vía helm con lo mencionado en el punto 2. Diseño de la solución. Finalmente se crearán reglas de alerta proactivas que serán evaluadas por Grafana Alerting sobre métricas de Prometheus para reflejar los incidentes de forma visual en los tableros.

Extra mile: Dado que las alertas de Grafana Alerting solo son visibles al entrar al panel, se propone configurar un punto de contacto de tipo Discord. Se crea un webhook en un canal del equipo y mediante una política de notificación, se envían hacia allí las alertas cuando se disparan. Grafana publica automáticamente un mensaje con la alerta, el servicio afectado y un enlace al tablero, y notifica también su resolución.

3.2 Fuera de Alcance:

Queda fuera de esta prueba la modificación, recompilación o mantenimiento del código fuente de los microservicios, la persistencia en algún almacenamiento de la nube y el despliegue de un clúster independiente de Prometheus AlertManager.

3.3 Casos de uso:

3.3.1: Detección de degradación del servicio mediante Reglas de Alerta:

- **Escenario:** Un microservicio experimenta una tasa anómala de respuestas erróneas que compromete la disponibilidad de la plataforma.
- **Breve descripción:** Se fuerzan errores 500 (*POST /chaos/status/500*) y se verifica que Grafana Alerting dispare la alerta y el panel muestre la información pertinente.
- **Objetivo:** El objetivo es mostrar un bajo tiempo de detección y la prevención de caídas silenciosas.

3.3.2: Identificación de cuellos de botella:

- **Escenario:** La plataforma presenta largos tiempos de respuesta y se requiere identificar qué componente o llamada interna genera la demora.
- **Breve descripción:** Se agrega latencia (*POST /chaos/latency/{ms}*) y se identifica en Tempo el span que genera la demora.
- **Objetivo:** El objetivo es identificar en pocos segundos si una demora proviene del frontend, de la red, de un microservicio específico o de la base de datos.

3.3.3: Identificación de causa raíz de un problema:

- **Escenario:** Se produce una excepción o error transaccional en un microservicio de backend y se requiere acceder a sus registros de depuración sin consultar pods manualmente.
- **Breve descripción:** Desde ese span se accede a los logs en Loki mediante tracesToLogsV2.
- **Objetivo:** El objetivo es mostrar cómo se puede obtener una correlación de logs con el problema sin necesidad de hacerlo de forma manual .

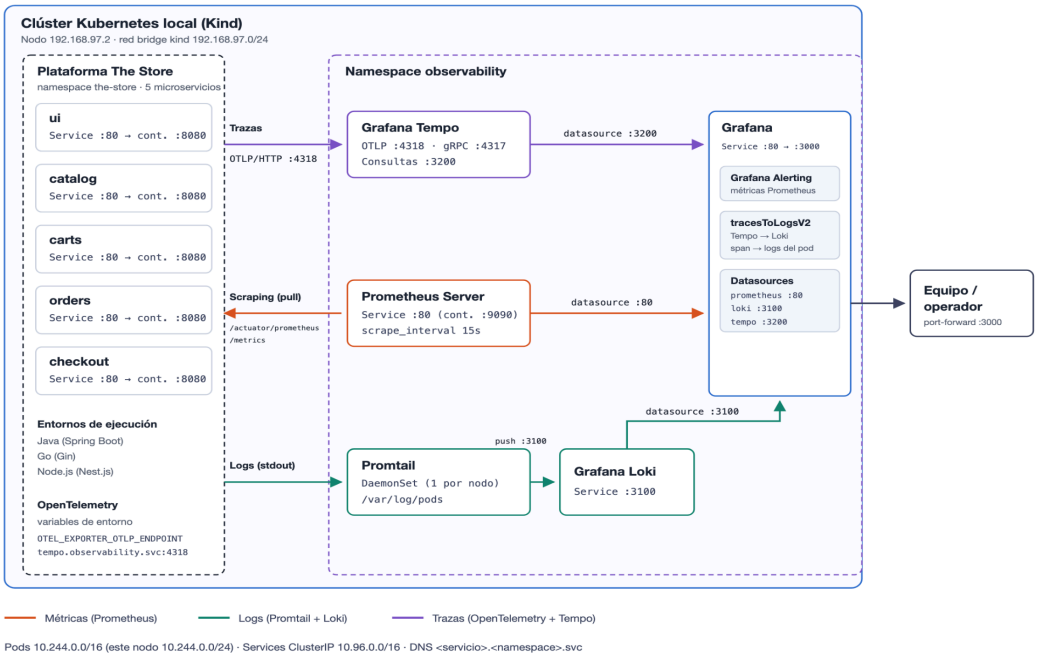
3.3.4: Visualización de información en Dashboard:

- **Escenario:** El equipo necesita una vista única del estado de la plataforma en lugar de consultar cada herramienta por separado.
- **Breve descripción:** Se arma en Grafana un tablero para la plataforma. Reúne el estado de los cinco microservicios y sus métricas RED (requests, errores y latencia). El tablero muestra además las llamadas entre servicios, los logs con error de Loki y las trazas con error o lentas de Tempo.
- **Objetivo:** Mostrar el estado de toda la plataforma en una sola pantalla.

3.3.5: Detección y diagnóstico de incidentes de infraestructura:

- **Escenario:** El sitio presenta tiempos de respuesta elevados sin evidenciar fallos explícitos en los servicios, haciendo preciso identificar si el origen del inconveniente radica en la capa de infraestructura del clúster.
- **Breve descripción:** Se incorpora en Grafana un tablero secundario enfocado en infraestructura para visualizar métricas de recursos en tiempo real, abarcando deployments, nodos y pods a partir de la información reportada por los propios contenedores. Para evaluar el impacto, se limita la CPU de una instancia y se monitorea la saturación junto con las reinicializaciones resultantes.
- **Objetivo:** Exhibir el procedimiento para identificar y resolver fallas a nivel de infraestructura en tiempo real, sirviendo de soporte complementario al panel de métricas de los servicios.

4. Diagrama



5. Alternativas consideradas

Zabbix se consideró porque fue mencionada como posibilidad en el enunciado y debido a que investigando se suele decir que es un estándar para monitorear infraestructura. Sin embargo, The Store ya expone métricas nativas en formato Prometheus, permitiendo su recolección directa sin instalar agentes ni programar exportadores adicionales. Además, Zabbix se enfoca únicamente en métricas, lo que implicaría sumar herramientas externas para logs y trazas. El stack de Grafana unifica los tres pilares en una sola interfaz.

Para las alertas, se tomó en consideración Prometheus Alertmanager. No obstante, dado que Grafana se utilizará para visualizar los tableros, se optó por su motor integrado (Grafana Alerting). Esta solución evalúa las reglas directamente sobre Prometheus y gestiona notificaciones desde la misma interfaz, logrando un resultado suficiente con un componente menos.